# Implementation Guide

Architecture notes, project conventions, implementation seams, and decision records.

**Working draft**

Architecture notes, project conventions, implementation seams, and decision records.

## Packet Runtime Modernization

## Status

Pre-reseed modernization chapter status. The early sections preserve the chapter history; the current state has seeded Definition/Bundle packet material, closed in-scope runtime genericization, packet-based policy/dependency semantics, retired legacy bridge write intents, and fortress-enrolled Preference.element writes.

## Summary

This chapter brings the packet system, generic fortress, runtime connectors, and mutation orchestration up to the Preference-as-template direction. Definition, Bundle, and Preference are now canonical packet types; Preference remains the working runtime example with a fortress-enrolled write path and runtime connector coverage for comparison. The broader goal is to keep every live packet type visible through the same coverage map while moving mutations into standardized trusted-local runtime paths.

The work should preserve current behavior while replacing hidden assumptions with typed registries, explicit missing coverage items, and tests that fail when the modernization map drifts.

## Modernization Targets

- packet types should have body schemas, compatibility entries, builder support, manifest definitions, definition parts, and runtime connector status recorded in one audit surface
- runtime mutations should map to prepare handlers, finalize handlers, policy action IDs, signed corridor use, and master-handler connector enrollment status
- Definition, Bundle, and Preference are canonical packet types with body schemas, compatibility entries, builder support, definition parts, and seed/profile coverage

- imported Definition and Bundle packets describe semantics but never introduce executable server behavior
- the generic fortress and master handler should become the standard runtime orchestration path after coverage is complete

## Phase Plan

1. Save the broad plan and add coverage audits. 2. Use the audit output to prioritize packet-type definition work. 3. Expand manifest definitions and definition parts type by type, preserving existing schemas and compatibility behavior unless a type-specific schema evolution is explicitly approved. 4. Adapt runtime mutation paths into runtime connectors behind the master handler, keeping signed corridor behavior intact until each mutation has a tested replacement boundary. 5. Enroll completed types and connectors only when their docs, tests, policies, and runtime behavior are all aligned. 6. Retire planned-gap records only when the implementation and tests prove the gap is actually closed.

## First Pass

The first pass is intentionally non-behavioral:

- add this chapter document
- link it from the implementation-guide index
- add packet type modernization coverage audits
- add runtime mutation modernization coverage audits
- add tests proving current types, mutation intents, Preference connector enrollment, and Definition/Bundle next-phase targets are all visible
- keep known modernization gaps green only through explicit planned-gap records

## Guardrails

- historical baseline: Definition and Bundle were initially kept out of PACKET_TYPES; the current promotion pass enrolls them as canonical packet types
- do not change MutationIntent, route payloads, packet schemas, or compatibility registry behavior during the audit pass
- do not migrate legacy scope-display caches until a dedicated compatibility pass
- the initial audit baseline kept Preference.element out of signed fortress enrollment; the current corridor now enrolls preference.element.set through prepare/finalize
- do not rebuild generated public docs artifacts as part of this first pass

## Test Plan

- run packet modernization coverage tests
- run runtime mutation modernization coverage tests
- keep packet-definition manifest and audit tests green
- keep fortress route and service-writer audit tests green
- validate docs with npm run docs:validate

## Decision Notes

The audit modules are the working checklist for the next implementation passes. Broken current invariants should fail tests. Expected gaps should remain visible as planned modernization work until they are closed by a later pass.

## Manifest Core Pass

The first chunky implementation pass expanded the packet manifest from the Preference template to the active packet types with generic builder-pipeline support: Element, Location, Role, Claim, Relation, Report, Proposal, Vote, Attestation, Decision, Action, Discussion, and Policy.

This pass remains runtime-ready. The new definitions describe existing body schemas, compatibility registry posture, generic builder support, action descriptors, planner descriptors, projection/index descriptors, and Definition parts. They do not change route payloads, packet schemas, runtime mutation behavior, or master-handler connector enrollment.

Builder-missing types remain explicit missing coverage items in the modernization audit. Preference later received canonical builder support and signed-corridor write enrollment.

## Packet-Type Authority Pass

The next implementation pass shifted the forward-looking checklist from legacy type enrollment toward manifest packettype authority. The later promotion pass enrolled Definition and Bundle as canonical packet types while preserving packettype language as the forward-facing semantic layer.

The pass added body builders for Definition, Bundle, and Preference. Those builders now feed canonical seed-profile helpers that create real Definition packet envelopes and a Bundle packet inventory for reseed material.

Packet-type modernization coverage is now the forward-looking audit surface for manifest definitions. The legacy type coverage remains as a migration bridge for live packet types and should keep missing coverage items visible until those types are converted into packet-type definitions and runtime connectors.

## Compatibility Definition Standard Pass

The compatibility standardization pass makes compatibility a required, auditable definition contract. Every manifest packet type now needs a required packet_compatibility Definition part and a current-version identity adapter descriptor.

Generic type definitions derive definition compatibility descriptors from the canonical compatibility registry. Current-only types expose identity compatibility, while legacy-aware types expose adjacent upcast/downcast ladder edges where the registry has adapter functions. Multi-step ladders use the fullchainbundle strategy so future bundles can carry discoverable adapter metadata without pretending every adapter must touch the current schema directly.

The manifest audit now fails when compatibility posture and descriptors disagree, when downcast edges lack loss awareness, when duplicate adapter edges exist, or when a claimed full-chain graph is disconnected from the current version. This keeps reseed and import/export planning honest before runtime handler extraction or generic fortress promotion begins.

## Fortress Handler Extraction Pass

The signed fortress corridor remains the live authority for prepare, proof, finalize, and persistence decisions. The packet-runtime master handler remains the client/API-to-runtime connector bridge; it does not own signed fortress internals yet.

The extraction pass introduces domain-composed fortress handler maps for locality, discussion, attestation, assembly, relation, role, and actor policy. MutationPrepareHandlers and MutationFinalizeHandlers remain compatibility facades for the current implementation, while the composed maps give the runtime a clearer stepping-stone toward generic packet planners.

Each live mutation intent now has a genericization classification:

- generic_ready means the intent is close to manifest/action/planner routing once its local read dependencies are isolated.
- plannerextractionneeded means reusable packet planning must be extracted before generic routing.
- workflow_specific means runtime orchestration still coordinates multiple packet operations or projections.

- legacy_bridge means the intent is a compatibility alias that should collapse into a canonical intent.

This pass intentionally preserves behavior. It records which fortress code should be retired, which should become reusable planners, and which orchestration remains runtime-owned before any live generic fortress promotion.

## Packet Operation Ontology Pass

The operation ontology pass adds the missing contract between packet definitions and trusted runtime execution. Packet definitions may now describe allowed mutation semantics by mapping their manifest mutation descriptors to known operation kinds such as singlepacket.create, singlepacket.revise, relation.set, claim.assert, attestation.set, bundle.import, projection.refresh, compatibility.adapt, and workflow.compose.

This ontology is an allowlist, not executable packet-defined code. Each operation records its expected planner kind, builder kind, result type, trusted local runtime engine, generic capability posture, and safety notes. Packet definitions can request known operation semantics, but only local trusted engines may execute builders, planners, adapters, workflows, or persistence.

The manifest audit now fails closed when a mutation descriptor cannot map to a known operation kind. The forward-looking packet operation modernization coverage lists every manifest mutation, the operation kinds it resolves to, the trusted local engine requested, and whether the gap is already mapped or still planned.

The signed fortress genericization audit also records operation mappings for every live mutation intent:

- generic_ready intents map directly to concrete operation kinds, but still wait for the later live promotion pass.
- plannerextractionneeded intents map to their target operation kind and keep an explicit planner extraction gap.
- workflow_specific intents map to workflow.compose or a composed operation set and remain runtime-owned until their component operations can be split safely.
- legacy_bridge intents point at the canonical operation direction they should collapse into.

This keeps the moat/drawbridge boundary intact. The runtime master handler can normalize client/API ingress requests and eventually choose operation descriptors, while the fortress still owns trusted prepare/finalize/proof/persistence until a later pass promotes selected operation kinds through the generic path.

## Generic Workflow Planner Contract Pass

The workflow planner contract pass adds the declarative layer above individual operation kinds. Packet definitions may now describe definition workflow plans as ordered steps over known generic operations, trusted resolver IDs, value bindings, simple conditions, policy action IDs, and runtime dependency IDs.

Workflow plans are data, not code. Definitions can say "resolve actor and target, then run relation.set" or "if the input value is present run attestation.set, otherwise run attestation.clear." They cannot introduce arbitrary functions, dynamic imports, persistence behavior, route payloads, or proof rules. The runtime interpreter validates every operation, resolver, dependency, condition operator, policy action, and step reference against local allowlists before producing a dry-run plan.

The first definition workflow plans cover the generic-ready fortress candidates:

- follows.relation.set
- follows.relation.clear
- role_association.claim.set
- attestation.packet_signal.set

These plans do not enroll live execution. They prove the manifest can describe packet-specific variables and ordered generic work while preserving the signed fortress as the only live prepare/finalize/proof/persistence authority.

Policy and dependency descriptors now matter as referenced workflow metadata, but their full semantics remain a dedicated pre-reseed pass. Unused legacy packet types remain explicit missing coverage items and do not block the generic workflow contract or switch-over planning.

## Workflow Alignment Pass

The workflow alignment pass connects the manifest workflow contract to the extracted fortress planner map. It adds a runtime-side audit that lists every live mutation intent, its genericization status, operation mapping, workflow-plan coverage, policy action IDs, trusted resolver/capability IDs, and remaining packet-specific assumptions.

The alignment map is the working checklist for retiring packet-specific fortress code. Generic-ready intents must have clean workflow dry-runs and trusted local capability coverage. Planner-extraction intents may either have a definition workflow plan or an explicit missing coverage item. Workflow-specific intents remain runtime-owned orchestration until their component operations can be split safely. Legacy bridge intents point at canonical workflow directions rather than receiving independent workflows.

This pass expands definition workflow coverage for knowable planner-extraction candidates:

- relation.association.add
- relation.association.clear

- home_locality.relation.set
- discussion.reply.create

The alignment remains runtime-ready. Existing runtime planner modules are registered as trusted local capabilities by descriptor, but their implementation is not moved, rewritten, or invoked through generic execution yet. Unused removed packet types remain visible missing coverage items and do not block switch-over planning.

## Runtime Crossing Guard and Fortress Handoff Pass

The crossing guard and fortress corridor remain separate layers. The runtime crossing guard owns client/API ingress normalization, manifest/workflow lookup, connector selection, definition handoff metadata, and response/refresh hints. The fortress corridor owns policy/proof authority, prepare/finalize lifecycle, mutation tickets, signed packet validation, persistence, and canonical mutation effects.

The handoff pass adds a definition PacketRuntimeFortressHandoff contract. A handoff records the normalized mutation direction, workflow alignment status, operation kinds, workflow plan IDs, trusted capability IDs, policy action IDs, dependency IDs, resolver IDs, fortress prepare/finalize handler names, and return refresh hints. It carries externaldefinitionexecution_enabled: false to record that imported definitions describe behavior while trusted local runtime code executes it.

Generic-ready and workflow-aligned planner-extraction intents can now produce definition_ready handoffs. Runtime-owned workflow intents produce explicit non-ready handoffs with orchestration reason codes. Legacy bridge intents point at canonical handoff directions. Unknown mutation intents fail closed before any fortress handoff.

At the time of this pass, this did not change the live mutation routes. The current state is stricter: NexusMutationService remains the live signed fortress authority, and authenticated Preference.element writes now enter that fortress service path rather than the old direct packet-runtime connector.

## Packet-Based Policy, Dependency, and Client Ingress Enrollment Pass

Policy and dependency requirements are now audited as packet-backed semantics rather than a second runtime-only dependency system. Workflow plans can reference policy action IDs and dependency IDs, but those references must resolve to packet policy semantics, packet Definition dependency parts, operation ontology entries, trusted workflow resolvers, or trusted local capability metadata. Runtime descriptors may index and validate those references, but they do not define packet meaning.

Policy packets remain the semantic authority for live write-lock policy. MutationPolicyGate remains the live resolver for scope policy refs, actor security mode, proof level, and accepted proof methods. Definition workflow policy descriptors record how policy action IDs map back to that packet-based enforcement model, while manifest-only actions remain definition metadata until a later live write-policy enrollment pass.

The runtime crossing guard now has a client/API ingress enrollment registry. The registry is an internal allowlist of adapter-originated transport routes and portable client intent IDs:

- /api/nexus/mutations/prepare enrolls the current signed fortress mutation intents.
- /api/nexus/mutations/prepare enrolls the Preference.element client intent; authenticated shell preference writes now use the same prepare/finalize corridor as other claimed mutations, while /api/nexus/shell-preferences remains guest compatibility state only.
- each enrollment records route or transport source, client intent ID, mutation intent, operation kinds, workflow plans, policy actions, dependency refs, and current live mode.

Unknown or custom route/intent pairings fail crossing-guard preflight. The preflight may resolve handoff metadata and packet-backed policy/dependency descriptors, but it does not authorize, ticket, sign, persist, finalize, or bypass the signed fortress. This keeps the future generic corridor aligned with enrolled client/API ingress from web, device, automation, or other adapters instead of accepting arbitrary injected operation requests.

## Interface-Neutral API Crossing Guard Pass

The enrollment layer is interface-neutral. Web shell, Raspberry Pi controls, local automation, and future adapters should all map their local events into the same portable client intent IDs, such as scope.follow.set, scope.association.clear, discussion.reply.create, and preference.interface.set. Runtime registries should not encode web UI concepts as packet meaning.

The live API routes now consult crossing-guard preflight before delegating to the live corridor:

- prepare parses the request intent, validates client/API ingress enrollment, then delegates to NexusMutationService;
- finalize reads the stored ticket, validates the ticket’s original mutation intent against enrolled prepare ingress, then delegates to NexusMutationService;
- authenticated shell preferences use the standard prepare/finalize mutation routes with preference.element.set;
- /api/nexus/shell-preferences remains a guest compatibility route and is outside packet-runtime connector enrollment.

This pass is still not generic execution. Preflight validates allowlist and metadata alignment only; fortress policy/proof/ticketing/persistence remains authoritative.

## No-Deferral Pre-Reseed Closure Program

Reseed design is now gated on full closure of in-scope live runtime modernization work. The chapter can still be split across multiple implementation passes, but the remaining live runtime work is no longer tracked as open-ended missing coverage items. The pre-reseed closure ledger classifies every live mutation intent, runtime connector path, workflow plan, policy/dependency requirement, client/API ingress enrollment, fortress handoff, and active packet type as closed, closingnow, queuedpre_reseed, or blocked.

The first proving promotion is follow relation set/clear:

- follows.relation.set
- follows.relation.clear

These intents now prepare through trusted generic workflow planning while NexusMutationService remains the signed fortress authority. API routes, route payloads, response shapes, policy action IDs, packet schemas, proof behavior, tickets, signatures, persistence, and projections remain unchanged. The promoted path uses manifest workflow metadata and trusted local relation planning to produce the same packet candidates and policy metadata as the previous fortress-specific follow planner path.

The remaining pre-reseed queue is explicit:

- relation, claim, and attestation generic enrollment for association, home locality, role claim, packet signal, and role attestation paths
- discussion and locality workflow decomposition for reply/thread planners, default surfaces, locality path/graph planning, and assembly creation
- packet-based policy/dependency semantic authority so Policy packets and Definition dependency parts carry enough meaning for reseed
- legacy bridge retirement for compatibility aliases that should not survive into the fresh reseed world, now closed by removing legacy bridge intents from the live prepare corridor
- a final reseed readiness audit after all in-scope modernization closure items are closed

Unused never-live packet types were pruned from active canon. They can return only through the same definition, schema, builder, seed, and audit path as any other active type.

## Generic Composite Workflow Adapter Pass

Complex graph workflows now use named trusted composite adapter shapes in definition. These adapters are local runtime-owned orchestration patterns that compose known packet operations, policy actions, dependency refs, and result metadata. Packet definitions and workflow alignment may reference adapter IDs, but packet definitions still cannot provide executable code.

The first adapter shape is composite.batch.packet_operations, used by locality.graph.apply. It describes the reusable graph pattern: resolve inputs, plan structural packets, plan relation operation batches, resolve grouped policy, prepare unsigned digests, carry prepared-result metadata, and classify projection/refresh side effects as runtime return extensions.

Two additional graph-style shapes are recorded for reuse:

- composite.defaultpacketset.ensure for idempotent default packet-set creation, first represented by discussion.surfaces.ensure.
- composite.entitycreate.withfollowups for a primary entity packet plus optional follow-up operations, first represented by assembly.element.create.

This pass remains runtime-ready. Live API routes, payloads, fortress ticketing, signing, persistence, projections, and response shapes remain unchanged. Complex workflows stay queued for live promotion until adapter parity tests prove prepare/finalize behavior against the current fortress oracle.

## Remaining Runtime Genericization Closure Pass

The second live generic promotion expands the trusted workflow seam beyond follow relations. The direct operation paths now enrolled behind NexusMutationService are:

- follows.relation.set
- follows.relation.clear
- relation.association.add
- relation.association.clear
- home_locality.relation.set
- role_association.claim.set
- attestation.packet_signal.set

The live behavior contract is unchanged: API payloads, policy action IDs, ticketing, signatures, packet schemas, persistence, projections, and finalize handlers remain the current fortress authority. The prepare side now resolves these direct operations through trusted local generic planners for scoped Relation, role Claim, and packet-signal Attestation writes, using the current fortress planners/builders as the behavior oracle.

The remaining composed workflows now have named adapter shapes instead of open-ended gaps:

- composite.locality_path.create.v0 for reusable entity/path creation and directory projection refresh.
- composite.discussionthreadpost.create.v0 and composite.discussion_reply.create.v0 for canonical Discussion(subtype: post/message) writes.

- composite.role_attestation.set.v0 for mutual-exclusion support/dispute/clear attestation composition.
- composite.actorwritepolicy.update.v0 for actor-owned Policy packet revision plus actor projection refresh.

Discussion follow-up is closed for the fresh canon: new top-level discussion writes use Discussion(subtype: post) semantics, while replies use Discussion(subtype: message). DiscussionThread, DiscussionPost, and DiscussionReply are not active fresh packet types.

Initiative follow-up is also explicit: the fresh-reseed direction is Action(subtype: initiative) as the default OWA anchor for policy, template, branding, locality, voting, and governance defaults. Cause is not an active fresh packet type. This pass does not add an initiative selector or UI behavior.

## Live Composite Workflow Promotion Pass

The runtime genericization lane is now closed for in-scope live prepare handling. Direct packet operations continue through the trusted generic operation seam, and composed workflows now prepare through trusted generic-composite workflow resolvers:

- composite.locality_path.create.v0
- composite.locality_graph.apply.v0
- composite.discussion_surfaces.ensure.v0
- composite.assembly_element.create.v0
- composite.discussionthreadpost.create.v0
- composite.discussion_reply.create.v0
- composite.role_attestation.set.v0
- composite.actorwritepolicy.update.v0

These resolvers execute trusted local runtime code only. Adapter descriptors describe the reusable workflow shape and audit metadata; packet definitions still cannot inject executable behavior. MutationPrepareHandlers remains the compatibility facade, NexusMutationService remains the signed fortress authority, and finalize handlers remain unchanged.

Actor write-policy update is mechanically promoted through the composite seam, and Policy packets plus Definition dependency parts are authoritative enough for the fresh genesis contract. Discussion canonicalization to top-level Discussion(subtype: post) plus reply Discussion(subtype: message) and OWA Action(subtype: initiative) are now part of the active reseed contract.

# Initiative Action Hierarchy and Discussion Schema Readiness

The pre-reseed packet model now treats Action(subtype: initiative) as the forward initiative/work hierarchy anchor. Action packets can carry hierarchy refs plus packet-backed policy, template, and default packet-set refs so OWA defaults can be overridden without adding OWA-specific fields to Element or hardcoding defaults in runtime.

Canonical discussion shape now reserves Discussion(subtype: post) for top-level multimedia forum artifacts that start a thread, while Discussion(subtype: message) remains the reply/comment shape. Legacy thread/post/reply packet types are pruned from fresh canon.

Governance hooks remain schema-ready rather than workflow-complete: quorum, minimum trust, voter eligibility, approval thresholds, and voting gates should be expressed through packet-backed Policy/default material linked from the applicable initiative Action, scope, proposal, or definition context. Decision is the formal outcome packet; Report(subtype: decision_report) is reserved for future tally/evidence/process closure material.

# Packet-Based Policy and Dependency Semantic Authority

Policy and dependency semantic authority is now closed for reseed readiness, while live governance execution remains later work. Policy current schema includes nullable defaultpolicy and governancepolicy sections. Older Policy revisions upcast those sections to explicit null; downcasts to older schema versions report loss when non-null default or governance material cannot be represented.

Policy packets are the semantic home for write locks, trust baselines, relation requirements, dependency and alignment rules, default inheritance, and governance hooks. The live write-lock path still runs through MutationPolicyGate; the new semantic helpers resolve and audit packet meaning without executing proposal/vote/decision behavior.

Definition packet_dependency parts now carry meaningful dependency refs for packet operations, builder pipelines, action bridges, canonical packet-type builders, Preference projections, Bundle inventory building, and trusted compatibility/projection seams. Workflow and runtime dependency IDs must resolve through one of these anchors, Policy packet semantics, the operation ontology, a trusted workflow resolver, or an explicit trusted local engine contract.

The seeded OWA Action(subtype: initiative) now links to default-inheritance and governance-baseline policies. Forward default/policy resolution uses the Action initiative anchor.

## Canonical Subtype Reset

The pre-reseed reset prunes inactive and legacy packet types from active canon. Fresh canon now includes only Definition, Bundle, Element, Location, Role, Claim, Relation, Report, Proposal, Vote, Attestation, Decision, Action, Discussion, Policy, and Preference.

Every active packet body uses top-level body.subtype as its packet classifier. Fresh writes reject old top-level classifier names such as kind, policykind, rolekind, proposalkind, claimkind, and attestation_kind. Nested rule mechanics can still use precise names such as quorum or threshold kind when they are not packet classifiers.

Cause, Signal, separate initiative/work types, separate discussion thread/post/reply/forum/space types, Minutes, Artifact, and other pruned types are not valid fresh packet types. The alpha database is expected to be archived and wiped rather than adapted into fresh canon.

## Final Pre-Reseed Wrap-Up

The final wrap-up retires the remaining live legacy bridge mutation intents from fresh writes:

- association.claim.set
- home_locality.claim.set

Canonical writes now enter through relation.association.add, relation.association.clear, and home_locality.relation.set. Historical legacy claim material remains readable/importable/projectable through compatibility surfaces, but the signed fortress prepare corridor, client ingress registry, handoff coverage, and live write-policy action list no longer enroll the legacy bridge intents.

The final readiness handoff lives in runtime audit code as createFinalPreReseedReadinessReport(). It records canonical write intents, compatibility-only legacy surfaces, OWA seed/default anchors, required default policies, discussion default packets, canonical definition packet types, and out-of-scope never-live packet types. Reseed design should start from that report rather than rediscovering chapter state from scattered modernization audits.

## Definition Packetization and Preference Fortress Promotion

Active manifest definitions now have canonical packet material. buildDefinitionPacketSeedEnvelopes() emits schema-validated Definition packet envelopes for every active manifest definition part, and buildDefinitionBundleSeedEnvelope() groups those envelopes into one Bundle.packet_set definition profile inventory. auditSeededPacketDefinitionProfile() compares that packet material back to the core manifest and fails on missing parts, duplicate or stale bundle refs, digest drift, or manifest/profile mismatch.

Definition, Bundle, and Preference are now first-class canonical packet types. The active definition profile is seeded as real packet material, but execution remains trusted-local: imported Definition or Bundle packets can describe schemas, operations, policies, dependencies, planners, and builders, but cannot introduce executable server behavior.

This is packetized seed truth, not imported-code execution. Stored Definition and Bundle packets may describe schemas, operations, policies, dependencies, planners, and builders; trusted local runtime registries remain the only executable authority.

Claimed Preference.element writes now use the signed fortress prepare/finalize path as preference.element.set. The client prepares through /api/nexus/mutations/prepare, signs the prepared Preference packet candidate, finalizes through /api/nexus/mutations/finalize, and then receives the same projected Preference result shape from the mutation result. /api/nexus/shell-preferences is now guest compatibility state only. The old direct preference.element.interface.set connector is retained as a definition/internal comparison bridge rather than the live claimed-write path.

# Architecture And Layers

## Product layers

The long-term architecture remains a three-layer stack with strict boundaries:

### OWA App

The public-facing civic surface.

- OWA-native framing and navigation
- geography-first assembly experience
- place -> assembly -> deliberation -> proposals or votes -> actions -> record -> learning

### Nexus Browser

The packet and graph surface.

- inspect packets
- follow links

- inspect provenance
- search and filter
- browse incoming and outgoing references

### Nexus Core

The portable engine with no UI assumptions.

- packet typing and validation
- graph building
- read and write operations
- import/export bundles
- merge strategy
- trust hooks
- policy evaluation

The load-bearing rule stays the same: data is the system; platforms are adapters.

## Repository boundaries

The active repo split is:

- core/* for portable packet logic, schemas, builders, interpreters, projections, and contracts
- runtime/* for persistence, runtime services, query services, auth/trust/discussion orchestration, and API glue
- app/* for application-layer components, hooks, constants, public content, and shared state
- src/app/* for the Expo Router route shell and API entrypoints

## Documentation system

The multi-chapter internal docs now use a chapter-first source-of-truth model.

Canonical content lives in:

- docs/implementation-guide/*
- docs/specifications/*
- docs/roadmap/*

The top-level files:

- docs/implementation-guide.md
- docs/specifications.md
- docs/roadmap.md

remain short local index shells only.

Public docs generation is now expected to follow that structure:

- docs/public/public-docs.manifest.json owns the ordered public source lists
- scripts/validate-public-docs.mjs validates manifest and shell/source-of-truth rules
- scripts/build-public-docs.mjs compiles readable docs data, Markdown downloads, PDF downloads, and version records
- generated artifacts under app/public/generated/, public/downloads/, and docs/public/version-records/ are derived outputs and should not be edited by hand

Expected workflow:

- update chapter files first
- run npm run docs:validate after canon doc edits
- rely on npm run docs:build during site/export builds or when intentionally refreshing generated public docs artifacts

## Core versus adapters

### Core owns

- packet schemas and validation
- graph relationships and traversal
- import/export/merge rules
- signing and verification law
- policy evaluation hooks and deterministic business logic

### Adapters own

- routing and navigation
- rendering and layout
- input UX
- device or browser integration
- orchestration around core-owned operations

Adapters should not invent parallel business logic for trust, validation, compatibility, or merge behavior.

## Navigation and shell model

The intended navigation model still uses three mental axes:

- place or scope

- topic or work
- time or evidence

The current implemented Nexus slice is a practical early version of that model, with:

- dashboard
- discussions
- votes
- roles
- trust
- library
- packet explorer as a shell-level inspection workspace
- account and identity custody as wrapper-level surfaces

# Core Entities And Packet Model

## Packet graph foundation

The packet graph remains the most robust unifying model.

Core packet rules:

- packet_id is the stable logical packet identity
- revision_id is the immutable exact revision identity
- parentrevisionrefs represent DAG ancestry
- schema_version tracks type-shape compatibility
- edges are the shared typed relationship collection
- authorityscoperef and applicablescoperefs stay separate
- revision_mode expresses type write semantics

Raw stored packets remain the historical signed fact. Adapted packets are runtime read views, and interpreted or read-model outputs are additional projections on top of those reads.

## Core entities

### Element

The universal identity anchor.

- person
- assembly

- organization
- service
- overlay or coordination group

Current direction:

- Element remains the durable actor, scope, and container type
- Element.subtype is the canonical classifier surface
- fresh Element packets reject the old top-level kind classifier
- dotted subtype forms such as person.claimed_identity or assembly.city are the forward model when the older shape previously depended on multiple classifier fields

### Scope

The context or lens around an element. Every person can be treated as their own scope lens through You.

### Assembly

A policy-governed civic or geographic element subtype, not a separate storage primitive.

### Initiative

A generic Nexus concept for policy, template lineage, defaults, and work hierarchy. The pre-reseed reset treats Action(subtype: initiative) as the canonical top-level initiative anchor above campaign, program, mission, and task semantics. Cause is no longer an active fresh packet type.

### Claim

The assertion and argument type.

Current forward direction:

- Claim can target packets generically
- Claim may carry claim_markdown and supporting refs
- Claim(subtype: relation_assertion) is the forward shape for claims specifically asserting a relation
- relation-specific claim semantics live under Claim(subtype: relationassertion).relationassertion.subtype

Current home-locality direction:

- canonical writes now use home_locality.relation.set
- that write produces Relation(subtype: home_locality) only; claims and attestations can be added around the relation, but are not automatically minted

- legacy homelocality.claim.set is retired from live fresh writes; historical Claim(homelocality) material remains readable through compatibility projections
- revise and withdraw semantics remain packet-native: status changes are represented by newly signed packet material rather than in-place mutation

### Attestation

The evidence, certification, support, dispute, and packet-signal type.

Current code truth:

- claim support and dispute currently stay on Attestation
- Attestation and Claim are still distinct in code
- Attestation.subtype is the canonical attestation classifier

### Role

A reusable role definition packet type. Exact-scope role assertions are currently represented through Claim(subtype: "relationassertion") with relationassertion.subtype: "role_association".

### Policy

The configuration and legitimacy type for defaults, thresholds, and later execution rules.

Current direction:

- actual adopted or followed graph facts belong in Relation
- asserted or disputable statements belong in Claim
- evidence and support/dispute posture belong in Attestation
- legitimacy-sensitive relation rules belong in Policy.relation_requirements
- default inheritance belongs in Policy.default_policy, using packet refs for policies, templates, default packet sets, and preference material rather than runtime-only default names
- governance readiness belongs in Policy.governance_policy, reserving voter eligibility, trust stage, quorum, approval, vote method, and decision-report hooks without executing voting yet
- dependency requirements remain in Policy.dependency_policy; subscriptions record what a subject accepts or excludes, and projections compare the two

### Decision

A governance artifact type that exists in infrastructure and read surfaces, but whose real workflow semantics are still developing.

## Converging civic grammar

The long-term Nexus direction is converging on a smaller reusable civic grammar:

- Element
- Relation
- Claim
- Attestation
- Report
- Action
- Policy

Current code already implements most of that grammar directly. Action now carries the forward initiative/work hierarchy and packet-backed default override refs, while Report has landed as a real packet type but its live use is still intentionally narrow.

Current code now ships the first concrete Report type use:

- Report(subtype: verification_report)
- Report(subtype: import_report)
- Report(subtype: decision_report) as reserved governance closure-report shape

This first live usage should still be understood narrowly. The type now exists as real packet infrastructure, but broader audit, incident, decision, or resolution report semantics remain later architectural work rather than implicit current product truth.

## Packet-backed defaults

Defaults should remain packet-native rather than becoming Element fields or runtime constants. The current pre-reseed inheritance direction is:

- packet definition defaults
- bundle/default packet-set material
- initiative Action policy/template/default packet-set refs
- element policy/template refs or relations
- actor/client Preference

OWA-specific behavior should be represented by the default OWA Action(subtype: initiative) packet and its linked policies, templates, bundles, and preferences, not by special cases in generic packet schemas.

The default OWA seed now links its forward Action(subtype: initiative) anchor to default-inheritance and governance-baseline Policy packets. New default/policy resolution should use the Action anchor.

## Schema evolution discipline

Before changing packet schemas, read this chapter first.

Also read docs/implementation-guide/trust-moderation-and-policy.md before changing Claim, Attestation, Relation, or Policy semantics.

For any packet type schema version change, the required checklist is:

- update the active schema or body shape
- update the type compatibility registry entry
- add or update upcast and downcast adapters where backward compatibility is intended
- update current schema version metadata
- update builders and type build definitions so new writes emit the canonical current shape
- update signature and write-preparation behavior if additive or defaulted fields affect compatibility or signing
- add or update tests for parse and read compatibility, adapted read behavior, write preparation, and any supported upcast or downcast path
- document the change in the relevant chapter and add a decision-log note when the change is architecture-significant

## Relationships and graph semantics

The graph should continue to express relationships through typed refs and edges rather than UI-only linkage. Important relationship categories remain:

- references or depends_on
- proposes or supersedes
- belongs_to
- scoped_to
- endorsedby or vouchedby
- implements or enacts
- reports_on
- forkof or derivedfrom

Current scope-graph direction:

- canonical mounted ancestry prefers Relation(subtype: defaultancestryparent)
- canonical home-locality projection prefers Relation(subtype: home_locality); legitimacy evidence can attach through separate Claims and Attestations when policy or contestation requires it

- canonical follows now use Relation(subtype: follows) and are actor-only; legacy shell follow preferences are compatibility-only read input
- canonical association now uses Relation(subtype: association) without automatic claim wrapping, and associated scopes now count as mounted related scopes in shell projection
- canonical policy adoption now uses Relation(subtype: subscribesto) targeting a Policy packet rather than a separate adoptspolicy relation subtype
- dependency requirements remain policy-layer semantics, usually Policy.dependencypolicy, while dependson remains available only as a structural edge type rather than a Relation subtype
- subscription relations can carry subscription_options for inherited/default policies, dependencies, modules, templates, and default packet sets; excluding a required default does not erase the subscription, but projection should report partial alignment or review needs
- Relation(subtype: definedbylocation) is the live read seam for linked Location packets
- locality.path.create now emits locality Element packets, defaultancestryparent relations, provisional Location(subtype: region) packets, and definedbylocation relations together
- locality rows can now carry dynamic descriptor metadata, which is currently stored in linked Location.spatialpayload.scopedescriptor rather than through a packet schema bump
- legacy locality levels such as nation | region | city | district now function as compatibility buckets, while actual ancestry comes from the ordered path graph and not from a hardcoded four-slot ladder
- locality depth remains projection-only and should not be stored as a universal packet truth
- legacy parentscope ancestry, shell follow preferences, and legacy Claim(homelocality) reads now belong in explicit compatibility projections or compatibility mirrors rather than inline main-path logic

# Trust, Moderation, And Policy

## Trust direction

Trust should remain layered, inspectable, and threshold-based rather than collapsing into a hidden score.

Current code truth:

- trust stages are currently explicit and threshold-based
- support and dispute evidence are packet-backed
- role posture is projected from scoped claims plus claim-targeted attestations

Direction:

- trust should stay scope-local
- trust should gate posting, voting, review, moderation, or role authority only through explicit policy and thresholds
- evidence and posture should remain inspectable even when defaults hide or deprioritize low-trust activity

## Moderation and automoderation

Status: canon candidate

### Current code truth

- moderation workflows are not yet fully implemented
- some read surfaces already expose packet-backed evidence and action visibility
- disabled placeholders are visible in several surfaces

### Direction

- automoderation should be a generic feature, not a route-local trick
- default moderation baselines should come from the relevant element’s policy
- user-level moderation preferences should be available as adjustable policy or settings choices, similar in spirit to write protection preferences
- moderation should primarily act as visibility projection over packet and attestation graphs before it becomes hard rejection logic

### Unresolved

- which moderation controls belong in generic policy versus OWA default policy
- which interactions should remain soft projection versus hard runtime enforcement
- how much moderation preference state should be actor-level policy versus client-level shell preference

## Attestations, reactions, and claims

Status: canon candidate

### Current code truth

- Relation is the structural type for adopted graph facts
- Claim is its own live packet type for assertions, arguments, and disputable relation claims
- Attestation is its own live packet type for evidence, certification, support/dispute, and packet-signal responses

- current discussion voting uses packet-signal attestations rather than a separate reaction type
- current claim support and dispute still run through the attestation service and attestation indexes

### Direction

- keep Claim and Attestation distinct
- treat Claim as the forward layer for relation assertions and other packet-targeted arguments
- treat Attestation as the forward layer for supporting, disputing, certifying, or signaling around packets, including Claims and Relations
- do not require Claim wrappers for every Relation
- let policy require supporting Claims for legitimacy-sensitive Relations where needed

Current implementation note:

- the new Policy.relation_requirements seam exists so stricter relation-support rules can be expressed generically instead of being hardcoded as route-only logic
- Policy.default_policy now carries packet-backed default refs for policies, templates, default packet sets, and preference material; it must not introduce runtime-only default labels
- Policy.governance_policy now reserves packet-backed governance hooks for voter eligibility, minimum trust stage, quorum, approval threshold, vote method, and decision-report expectations
- this chapter should be read before changing Claim, Attestation, Relation, or Policy semantics because it owns the intended separation between assertion, evidence, graph structure, and policy requirements
- packet-native follow does not currently require a supporting claim in this phase
- packet-native association no longer auto-mints a supporting self-issued Claim(subtype: relation_assertion) alongside the structural relation

Current home-locality policy note:

- OWA-sensitive home-locality legitimacy resolves through Action(subtype: initiative) as the forward OWA policy/default anchor
- a canonical Relation(subtype: homelocality) is enough for default mounted home-locality projection; stricter Policy.relationrequirements can still require extra evidence for specific scopes
- the expected support model in this phase is separate Claims and Attestations attached around Relations only when evidence, dispute, or policy asks for them
- legacy claim-only home-locality reads remain compatibility projections, not the forward legitimacy model

Current dependency authority note:

- Definition packet_dependency parts describe packet requirements and local engine contracts; runtime registries only validate and interpret those refs
- workflow dependency IDs must resolve to a Definition dependency part, Policy semantic, operation ontology entry, workflow resolver allowlist, or trusted local engine contract
- trusted runtime capability metadata is allowed only when it points back to packet meaning or an explicit local engine contract

## Longer-term civic review framing

Direction:

- Claim should be understood as an unresolved assertion or request that invites evaluation, scrutiny, or action
- Attestation should be understood as a signed stance toward a target, such as support, dispute, certification, rejection, or abstention
- a Vote should remain a governed attestation inside a recognized process with eligibility and policy rules
- Report should be the long-term home for findings, outcomes, and context around a target
- a resolution or decision artifact should eventually read as a process-backed closure report rather than as unexplained authority
- quorum, minimum trust, eligibility, approval thresholds, and voting gates should be policy-backed defaults that can be inherited from the applicable initiative Action or scope policy and overridden by explicit packet refs

This framing is architectural direction, not a statement that all of those packet types or workflow surfaces already exist as live runtime behavior.

## Verification truthfulness direction

Future verification UI and policy language should distinguish at least:

- structurally valid
- cryptographically verified
- provenance known
- locally trusted
- substantively true

Current product language does not yet fully expose those distinctions. The next verification/reporting work should avoid collapsing them all into one vague verified concept.

Current verification chapter truth:

- the runtime now signs verificationreport and importreport packets with a normal local validator identity
- those report packets are the signed record of what the node concluded
- a local runtime-owned verification index/cache exists so dashboard cards, Explorer search rows, export lookup rows, action menus, and Explorer verification views can project truthful current status quickly
- imported external verification reports are readable evidence, but local trusted status still comes only from the latest local validator report in this phase

# Governance, Initiatives, And Decisions

## Initiatives

Status: canon candidate

### Current code truth

- initiative-specific filtering, subscriptions, and version visibility are not yet active product behavior
- current code does not yet implement initiative-version subscriptions or official versus unofficial filtering

### Direction

- Action(subtype: initiative) is the forward top-level initiative packet for policy, template lineage, default packet sets, and later work hierarchy
- OWA should be modeled as an initiative Action inside Nexus, not as a hardcoded exception
- Cause is no longer an active fresh packet type
- lower work hierarchy levels should be represented through Action subtypes such as campaign, program, mission, and provisional task
- "official OWA" should mean conforming to recognized OWA dependencies, templates, and policies
- dependency requirements remain policy-layer semantics; depends_on is not a Relation subtype
- policy adoption is modeled by Relation(subtype: subscribesto) targeting a Policy, not by a separate adoptspolicy relation subtype
- assemblies remain valid Nexus objects even when they fork or diverge from canonical OWA lineage

### Unresolved

- the exact packet/API shape for initiative conformance and recognition beyond Action refs and policies
- how official, unofficial, derived, and forked initiative states should be encoded

## Initiative versioning and subscriptions

Status: canon candidate

### Direction

Initiative lineage should support version-aware adoption and viewing, including:

- current official version
- older official versions
- upgrade availability
- optional inclusion of unofficial forks

Subscriptions and default visibility should eventually support choices like:

- current official only
- all official versions
- official plus unofficial
- multiple initiatives together
- inherited default policies/dependencies with explicit deselection and visible partial-alignment states

### Unresolved

- the exact version packet or metadata shape
- whether subscriptions belong primarily to actor policy, shell preference, or both
- how version compatibility and upgrade prompts should surface in Explorer and feeds

## Assemblies, custody, and legitimacy

Status: canon candidate

### Direction

- assemblies are elements with policy-governed civic semantics, not owned subsidiaries
- custody, legitimacy, policy authority, and governance outcome should remain distinct concepts
- association should remain a simple claim first; typed categories are unsupported

- multi-key assembly authority, custody transition, and protected-state mutation should remain explicit later design work

### Unresolved

- the exact custody and multi-key packet model
- how authority grants or keyset policies should be represented
- how passed decisions eventually authorize protected state changes

## Proposals and decisions

Status: canon candidate

### Current code truth

- votes remain read-only
- proposals, votes, and decisions are not yet a fully interactive trusted workflow

### Direction

- decisions should begin as legitimacy records and civic signals
- policy may later allow some decisions to activate real effects
- proposals should declare their intended outcome as explicitly and programmatically as possible
- quorum, eligibility, minimum trust, approval thresholds, and voting gates should come from packet-backed Policy defaults attached through the applicable initiative Action, scope, or proposal context rather than from hardcoded Vote fields
- a Decision should remain the formal outcome artifact, while Report(subtype: decision_report) is reserved for evidence, tally, and process-context closure material

Proposal outcome categories should eventually include:

- policy adoption
- mission or coordination plan
- resolution
- signal or poll
- other explicit action packages

Automatic execution should only be described as future behavior for explicitly supported built-in action handlers or canonical approval artifacts.

### Unresolved

- which decision types can become effect-bearing first
- how multi-key assembly authority interacts with execution

- whether execution produces direct mutations, canonical approval packets, or both

# Decision Log 2026-04

This monthly log condenses the April 2026 decisions that remain most important for current implementation and future migration work.

## 2026-04-08 foundations

- Packet identity uses stable packetid plus immutable revisionid.
- Revision history is a DAG with parentrevisionrefs, not a single chain.
- Packet relationships normalize into one typed edge collection.
- Element remains the identity-root type for people, assemblies, organizations, and services.
- The dedicated Nexus shell lives under /nexus/*.
- Function-first and scope-first remain one system rather than two route trees.

## 2026-04-09 shell and discussion shape

- Shared browser and Nexus query services over the packet graph became the projection seam.
- Active shell and scope routes moved onto packet-backed payloads instead of mock arrays.
- Discussions became a connected forum shell with feed, thread, and post workspaces.
- Feed cards became the primary thread-launch surface.
- Cursor paging and reply-branch controls became first-class route behavior.

## 2026-04-10 identity and auth

- Every graph-writing actor became a cryptographic Element(kind: "person"), including guests.
- Claimed auth uses local keys plus server challenge sessions rather than server-owned identity truth.
- Claimed auth now supports passkeys, rotating sessions, and explicit write-approval modes.
- Identity ceremonies moved into the Nexus shell.
- Attestation replaced packet votes as the reusable trust edge.

## 2026-04-17 architecture and trust pivot

- The final source roots became core, runtime, and app, with src/app as the Expo Router shell.
- Trust replaced Account as the top-level function surface.

- You became a first-class personal scope lens.
- Roles became a dedicated scope workspace.
- Role, Claim, and packet compatibility infrastructure became first-class runtime concerns.

### 2026-04-18 to 2026-04-20 locality and identity hardening

- Legacy signed identities remained valid after the roles-and-claims refactor.
- Home locality became the mounted geographic truth, while follows became shell preferences.
- Locality search and locality creation hardened around canonical geographic assembly flows.
- Location disclosure remained separate from mounted home-locality truth.

### 2026-04-23 and 2026-04-24 compatibility and fortress corridor

- Schema compatibility moved into a real raw/adapted/read-for-write pipeline.
- Fortress pass 1 moved discussion write admission into core mutation law.
- Fortress pass 2 introduced the shared prepare/sign/finalize corridor and packet-backed write-lock policy updates.

### 2026-04-25 to 2026-04-29 packet floor and verification

- Packet compatibility coverage is now classified explicitly.
- Legacy write seams were sealed before discussion-type cleanup.
- Raw packet signatures verify before adaptation.
- Live and near-live packet types now share one generic builder floor.

### 2026-04-30 surface-level runtime contracts

- Discussion workspaces now project action affordances through the shared NexusActionState and NexusActionIntentDescriptor contract.
- Packet Explorer became a shell-level read-only inspection workspace.

## Decision Log 2026-05

### 2026-05-19 initiative action and discussion schema readiness

- Pre-reseed initiative semantics now point to Action(subtype: initiative) as the forward top-level initiative/work hierarchy above campaign, program, mission, and provisional task semantics.
- Action(subtype: initiative) is the fresh-reseed initiative/default anchor; Cause is pruned from active canon.

- Canonical discussions now reserve Discussion(subtype: post) for top-level forum artifacts that start threads, while reply chains use Discussion(subtype: message).
- Governance schema hooks remain policy-backed: quorum, minimum trust, voting gates, and decision-report closure semantics should be expressed through linked Policy/default material, Decision, and Report(subtype: decision_report).

This monthly log condenses the May 2026 decisions that remain most important for current public-surface and documentation shape.

## 2026-05-01 public surface convergence

- Shared public actions and section shells now route through one surface contract.
- PublicCardFrame separates shared graphic treatment from page-owned content layout.
- Support and Docs now share the same public frame and panel system.
- Public card frames gained ambient generated backgrounds owned by the shared frame layer.
- About surfaces joined the shared public frame path without absorbing card-specific animation rules.
- Public sections now use the universal card animation path.
- Public theme cleanup centralized lingering route-local styling values into named public seams.

## 2026-05 chaptering follow-up

- Canon docs are being split into index files plus shallow chapter folders to reduce edit cost, preserve top-level link stability, and keep current truth, durable architecture, and roadmap work separated cleanly.

## 2026-05 docs workflow cleanup

- The multi-chapter internal docs now treat chapter files as the canonical content source, while the top-level files remain short local index shells only.
- Public docs generation for implementation guide, specifications, and roadmap now compiles from chapter files only instead of concatenating the top-level shell files into the public artifact.
- The docs build now validates chaptered manifest entries, missing or duplicate source files, generated-artifact misuse, and shell-file drift before producing readable or downloadable outputs.

## 2026-05 shell honesty foundation

- Nexus now uses a shell-level early-access gate as a session-scoped entry warning rather than leaving public-facing instability as unstated product lore.

- The gate is intentionally lightweight and UI-owned: it is not packet-backed, not actor-specific, and not yet a tutorial flow.
- The same shell seam should later be able to host onboarding, release notes, or other blocking notices without redesigning the overlay stack.

## 2026-05 feature-status explainers

- Disabled and partial Nexus controls now route through a centralized feature-status registry instead of scattering one-off placeholder copy through route files.
- The first-pass status taxonomy is intentionally small: comingsoon, readonly, partial, and custom, with registry-owned overrides when a control needs more specific wording.
- Nexus shell now hosts one explainer card overlay at a time so disabled controls can stay compact while still opening truthful contextual explanations above Explorer, Library, Votes, and future surfaces.
- The shared NexusActionButton seam remains semantically disabled for these controls, but can now open an explainer instead of acting like a dead button when a feature-status id is attached.

## 2026-05 Explorer and shell mobile cleanup

- NexusInlineSelect now renders its open menu on a true overlay layer instead of trusting local stacking order, so Explorer View as menus can sit above adjacent header controls.
- Packet Explorer tab decks now behave like normal wrapped rows until they exceed three rows, and only then become scrollable.
- Narrow-screen shell entry now prioritizes immediate access to the real menu rails by opening both rails when the tray is opened, instead of preserving collapsed desktop rail state inside the mobile tray.
- That same mobile tray now sizes itself to the currently visible rails and closes entirely when both rails are minimized.
- Library packet highlighting is now treated as Explorer-linked state rather than a permanently sticky URL-only cue, so stale selection clears when the corresponding Explorer packet tab disappears.
- Explorer resize now lives in a dedicated Explorer-specific desktop drag controller rather than the main overlay coordinator, and uses global window mouse tracking plus temporary drag capture instead of a moving PanResponder handle.
- Shared browser packet projections now route through the core packet-title projection path instead of falling back directly to raw packet ids, which keeps Explorer and other packet-inspection surfaces from flipping loaded titles into percent-encoded route strings.
- Explorer reading now uses one primary content scroll surface with local collapsible header bands, while Close tabs has moved out of the shell command row and into the packet-tab deck as a packet-management utility.

- Explorer collapse controls were then tightened so the bands do not reserve their own chevron rows: collapsed sections disappear fully, and their restore actions move into the top shell command row as Packets and Views.

## 2026-05 Packet Explorer export workbench

- Packet Explorer Home is now explicitly split into Search and Export sub-tabs so future Explorer capabilities can grow from one stable workspace without collapsing back into the main packet inspector.
- The first live portability seam is export only: Search stays present but non-live, while Export becomes the active raw-json workbench.
- Packet export defaults to Raw packet, which means the current preferred revision envelope only; bundle wrapping is an explicit opt-in instead of the default artifact shape.
- Bundle export remains additive and transport-oriented: the envelope now records export metadata such as mode, root refs, counts, title, and note without changing packet schemas or mutating the packet envelopes inside packets.
- Phase 1 bundle scopes stay concrete and bounded to current graph/scoping contracts: packet history, outgoing references, incoming referrers, scope-stack ancestry, their combined union, and full local-store export as a separate node-level snapshot tool.
- Preview and download now share one export builder seam with explicit size guardrails so Explorer can show inline JSON only for small payloads while still allowing larger .json bundle downloads without inventing a second export format.

## 2026-05 Packet Explorer import workbench

- Packet Explorer Home now expands to Search, Import, and Export, with Search-card import shortcuts routing into the new Import workspace rather than spawning separate packet-vs-bundle pages.
- Import is intentionally a two-step flow: Analyze first, then Commit, so pasted or uploaded JSON does not mutate the local store until the runtime preview says the payload is structurally safe.
- Phase 2 import stays generic and backward-tolerant by accepting raw packet envelopes, exported bundle envelopes with packets, legacy bundle envelopes with revisions, and raw arrays, all normalized onto the existing bundle-import substrate.
- Web import now includes a browser .json file picker, but paste remains the universal fallback so the workflow still works outside the browser without adding a cross-platform file abstraction in this pass.
- Import analysis is structural rather than trust-verifying: it reports invalid entries, duplicates, missing parent revisions, type conflicts, affected packets, and likely open targets, then blocks commit on unsafe inputs instead of offering partial-import toggles.

- Post-import repair now preserves local preferred-head intent when divergence appears: if a newly imported branch creates multiple heads and the old preferred head still exists, Explorer restores that preferred revision instead of silently replacing it with the last imported head.

## 2026-05 Packet Explorer live search and export lookup

- Packet Explorer Search is now a real manual discovery workspace instead of a placeholder card, and its state stays Home-owned so query text, grouped results, and the active category survive normal Explorer tab movement while the session stays open.
- Search intentionally stays preferred/current only in this pass: it reads from the shared packet search index rather than adding FTS, graph traversal search, or historical revision fanout into the main result set.
- Exact revision-id search is still supported through a separate revision resolver seam on the packet store, which lets Explorer map historical revision IDs back to the owning packet while keeping the displayed packet card anchored to the current preferred revision surface.
- Search results are grouped into Direct, Name, and Text, with deterministic ranking and lightweight Open / Export routing instead of turning the first live pass into a broader action hub.
- Export now includes a compact active lookup when no packet target is preloaded, but that lookup intentionally remains narrower than the full Search workspace so Explorer keeps one discovery-oriented search surface and one operational packet-picker.

## 2026-05 Packet Explorer workspace cleanup

- Home workspace navigation has now moved out of the Home body and into the shared inspector row, so Search, Import, and Export behave like first-class Explorer modes rather than nested content tabs.
- The packet primary rail and the Home workspace rail now share one compact attached-tab treatment, dropping the previous secondary tab copy so the inspector band can stay horizontally accessible without running off-screen.
- Home-only dead controls were removed instead of being left as placeholder chrome, and the Search workspace now keeps only its core manual finder actions rather than duplicating Import navigation through extra shortcut buttons.
- Packet export was re-centered around direct packet lookup when no target is preloaded, with the packet-picker promoted ahead of explanatory copy so the export workspace feels operational instead of instructional.
- Search now defensively tolerates a missing runtime revision resolver by falling back to the preferred/current packet index path instead of failing the whole search request.

## 2026-05 Packet Explorer final polish

- Link traversal in Explorer is now explicit instead of ambiguous: grouped links expose Open in new tab, Open in current tab, and View in Library, while current-tab retargeting preserves the existing inspector state and refreshes the active packet identity fields.
- Packet tab decks now present Home as the workspace tab label, use middle truncation for packet titles, and surface desktop hover metadata so long packet titles and revision context stay discoverable without widening the deck.
- Home Export now includes an explicit Cancel reset path for packet-scoped export so preloaded packet targets can be cleared without leaving the Export workspace or disturbing the separate full local store export surface.
- Search preview caps and category browsing are now separated: All remains an 8-result grouped preview surface, while focused Direct, Name, and Text views page through larger result sets with server-backed 25-item pages.

## 2026-05 Dynamic scope graph consumer pass 1

- Home locality is now relation-first in the mutation corridor: canonical writes use homelocality.relation.set and produce Relation(subtype: homelocality) without automatically minting supporting claims.
- Legacy homelocality.claim.set, legacy Claim(homelocality), and legacy parent_scope ancestry remain readable only through named compatibility adapters and projections rather than through mixed main-path shell logic.
- Mounted scope projection now prefers packet-native structural relations such as defaultancestryparent and definedbylocation, while also exposing richer packet-backed shell metadata for later UI graph and dashboard work.
- OWA home-locality projection now resolves from packet-native relations sourced through the forward Action(subtype: initiative) anchor path; relation requirements remain available for stricter policies but are not the default fresh-write wrapper.

## 2026-05 Scope graph hardening and packet-native scope writers

- Scope ancestry resolution now explicitly surfaces structural graph state, including compatibilityparent, conflictingparents, cyclicancestry, and missingparent, instead of silently treating all non-canonical cases as equivalent.
- Follow is now canonical and actor-only: new writes use follows.relation.set / follows.relation.clear, while shell cookie follows remain a compatibility read bridge only.

- Association is now relation-first in the mutation corridor: canonical writes use relation.association.add / relation.association.clear, keep the supporting self-claim layer, and mount associated scopes as related mounted scopes rather than leaving them as badge-only side state.
- The first packet-native shell UI pass keeps the current rail layout but now groups scope context into Home, Associated, Followed, and Discoverable sections, roots the home trunk at You, and routes follow/associate sidebar actions through the canonical mutation corridor with a refetched shell projection afterward.
- locality.path.create now emits locality Element packets, canonical defaultancestryparent relations, provisional Location(subtype: region) packets, and definedbylocation relations in one signed packet batch, while parent_scope remains a named compatibility mirror only.

## 2026-05 shared packet-card actions and focus

- Dashboard preview lists and focused packet panels now share reusable packet action menus instead of route-local action clusters.
- Dashboard preview surfaces can focus one packet at a time locally, letting the focused presentation reuse the same packet action vocabulary without inventing a separate packet-detail route.
- Compact tooltip-capable icon badges are now part of the shared Nexus packet-card language instead of being dashboard-only decoration.

## 2026-05 sidebar follow-up polish

- The Home scope trunk now renders broadest-to-smallest, with You at the personal end rather than as the visual root.
- Sidebar rows no longer depend on inline badge clutter or left-side tree graphics for their primary meaning.
- Compact side overflow menus are now the active scope-row action pattern, with section placement carrying most of the relationship signal.

## 2026-05 Explorer truth clarification

- Packet Explorer Search, Import, and Export are live and should be treated as current product truth.
- Explorer remains read-only for packet mutation.
- Verification is now live for local packet validation, import reporting, and Explorer verification surfaces, while richer peer trust, provenance weighting, and broader report semantics remain later seams.

## 2026-05 verification chapter 1

- Report is now a real packet type in code, with verificationreport and importreport as the first live subtypes.
- The local runtime now bootstraps a normal signed validator identity and uses it to sign packet verification and import reports instead of introducing a privileged core-signature bypass.
- Packet Explorer now has a dedicated Verification rail, while dashboard and shared packet action menus can Validate, Revalidate, and View verification.
- Import now supports Validate first, Validate after, and Don’t validate, with validation reports and packet-level verification summaries recorded through runtime services and cached locally for truthful UI projection.

## 2026-05 verification hardening and bundle direction lock-in

- Packet-signature verification now accepts a narrow explicit set of identity-bearing signer packets, including Element(kind: person) and Element(kind: service), so the local validator stays a real service identity instead of a fake person-only compatibility hack.
- Verification summaries are now revision-anchored: local cache rows record the exact target revision and digest they verified, and current verified state only applies while that preferred revision still matches.
- Unknown signer status is now intentionally truthful: a packet can remain signed while still being unverifiable on the local node when the signer packet is unavailable, instead of being mislabeled as cryptographically valid.
- Explorer Import now exposes recent import_report history and richer artifact metadata through the existing report packet type rather than introducing an interim import-history packet type.
- The future bundle direction is now locked in at the roadmap level: Bundle will become a first-class packet type later, rebundling will create a new bundle packet rather than revising someone else’s bundle, bundle history should behave as a lineage graph, and legacy bundle JSON remains a runtime transport artifact rather than a core packet-compatibility type.

## 2026-05 verification tidy polish

- Explorer verification now surfaces freshness explicitly: the current preferred revision, the revision targeted by the latest local report, and whether that report is current, stale, or absent.
- Import History now speaks honestly about its current model: it shows the latest local import report per artifact identity rather than pretending to be a full append-only event ledger.

- The next major implementation chapter remains locality UX and geo standardization; a dedicated validation workflow screen and first-class Bundle packets stay intentionally unsupported for later work.

## 2026-05 locality UX pass 1

- /nexus/locality/create now has a true non-mutating Review path seam instead of pretending the write corridor itself is the review step.
- Top-level locality search now activates the existing create_candidate handoff so missing places can seed the broad-to-narrow builder directly, including from the Enter key path.
- Duplicate warnings in locality review are now actionable through Use existing, Edit name, and Create anyway rather than appearing only as passive blocker text.
- Use as home locality is now an explicit shared toggle for both selecting existing localities and creating new ones.
- The home-branch inclusion checklist is now visible during review as a UI-only preview affordance, while authoritative storage and projection of selected included scopes remains unsupported to a later locality pass.

## 2026-05 locality foundations phase 2A

- Locality planner semantics are now ordered-path first: the runtime interprets locality preview and create input as a broad-to-narrow path, and sparse paths no longer need to fill every legacy ladder slot.
- The current locality type picker is now real runtime input instead of decoration: path rows can carry LocalityScopeDescriptor data, with safe fallback descriptors derived from current legacy buckets when callers omit them during rollout.
- New locality writes now persist descriptor metadata inside linked Location.spatialpayload.scopedescriptor, while legacy nation | region | city | district values remain compatibility buckets rather than the forward locality ontology.
- Locality search and duplicate projection now preserve Unicode scripts, normalize accents safely for matching, and keep useful legacy ASCII-style aliases where that compatibility bridge is feasible.
- Home-tree inclusion persistence and projection fields remain intentionally unsupported to the next locality foundations pass rather than being bundled into descriptor and ancestry work.

## 2026-05 locality hardening pass 2A.5

- Locality preview and create now reject decreasing legacy compatibility bucket order such as city -> nation, while still treating ordered path ancestry rather than the ladder itself as the real graph truth.

- Descriptor reads now validate hierarchy_system against the allowed runtime enum so malformed imported Location metadata falls back safely instead of being trusted as-is.
- Top-level locality search can now still route into creation when same-name results exist in a different branch, preventing one existing Ontario from suppressing a new Ontario under another parent path.
- The create page now clears stale carried-over target rows more carefully when the target type changes across legacy buckets, and successful create/reuse completion resets the create builder before the success modal appears.
- Trust home-locality presentation has been tightened into a smaller onboarding-friendly card without changing the underlying home-locality actions or relation-first write path.

## 2026-05 locality runtime catch-up chapter

- Locality confirmation now routes through one composite locality.graph.apply mutation intent above locality.path.create, so structural locality writes, selected home-locality changes, scope associations, follows, and display preferences are coordinated together instead of being chained from the client.
- Partial success is now explicit at that runtime seam: structural locality planning and packet writes remain phase one, while relation and display-preference writes report their own outcomes without pretending the whole flow is all-or-nothing.
- Actor-to-scope relationship reads are now centralized through one runtime controller that treats canonical home_locality, association, and follows relations as the main truth, while preserving guest and compatibility follow behavior as an explicit fallback bridge.
- main is a visible-scope preference rather than a relation, and associated/followed parent-context display toggles are persisted alongside it through the current claimed-actor preference bridge.
- The generic scope-graph projection now returns server-projected home, associated, followed, main, and discoverable sections for the sidebar, and OWA-specific initiative-anchor relation-policy lookup has been moved out of the generic graph core into a narrower adapter layer.

## 2026-05 Preference.element packet write corridor

- Preference is now enrolled in the canonical packet ontology as a replaceable packet type, with Preference.element carrying actor-owned scope-display preferences.
- Claimed actor scope-display writes now create live Preference.element packet revisions and also update the legacy runtime preference table as a compatibility cache.
- Scope-display reads prefer the latest active Preference.element packet and fall back to the legacy table when no packet exists; guest preferences remain cookie/session compatibility state.

- Claimed shell preference reads now resolve the actor from the authenticated session; actor query mismatches use guest projection instead of reading another actor’s private preference packet.
- Preference.element now includes an interface.shell_chrome section for navigation mode, theme mode, and UI density so remaining shell-interface preferences can move into the same body without redefining the packet shape.
- runtime/nexus/server/packet-runtime-master-handler.ts is the first generic runtime connector corridor: routes can request a connector id, the handler resolves the packet definition and mutation action plan, then dispatches to local trusted connector code instead of hardcoding route-specific packet writes.
- preference.element.interface.set was initially enrolled in that master handler as a live bridge. It wrote Preference.element.value.interface.scopedisplay, could preserve or update interface.shellchrome, and kept the legacy scope-display table as a compatibility cache. The later fortress promotion pass moves claimed writes to preference.element.set and keeps this connector runtime-ready.
- The manifest definition fortress bridge remains non-executing and descriptor-only. The new runtime master handler still runs trusted local connector code; it does not execute arbitrary behavior from imported definitions.

## 2026-05 policy/dependency semantic authority

- Policy current schema now includes nullable defaultpolicy and governancepolicy sections so default inheritance and governance readiness are packet-backed before reseed design.
- Dependency refs in Definition parts and workflow plans now resolve through packet dependency semantics, Policy semantics, operation ontology entries, trusted workflow resolvers, or explicit local engine contracts instead of loose runtime-only strings.
- The seeded OWA Action(subtype: initiative) is now the policy/default anchor and links to default-inheritance and governance-baseline policies.

## 2026-05-20: Canonical subtype reset before reseed

- Active packet canon is pruned to Definition, Bundle, Element, Location, Role, Claim, Relation, Report, Proposal, Vote, Attestation, Decision, Action, Discussion, Policy, and Preference.
- Cause, split discussion types, split initiative/work types, Signal, Minutes, Artifact, and other unused alpha types are removed from fresh packet canon.
- Fresh active packet bodies use top-level body.subtype as the packet classifier. Old top-level classifier names are treated as alpha archive shapes, not fresh write compatibility obligations.

- Reseed continuity will preserve identity/key continuity by default; stale packet-id relations, home locality, follows, associations, main-tree scope IDs, and old discussion placement are not carried forward by default.

## 2026-05 final pre-reseed wrap-up

- Live fresh writes no longer accept association.claim.set or home_locality.claim.set; canonical association and home-locality writes now use relation-first mutation intents only.
- Legacy claim/home-locality material remains compatibility-readable and importable, but it is no longer a live mutation corridor entrypoint.
- A final pre-reseed readiness report now inventories canonical write intents, compatibility-only surfaces, OWA default anchors, required seed policies, discussion defaults, canonical definition packet types, and out-of-scope packet types for the separate reseed design pass.

## 2026-05 definition packetization and Preference fortress promotion

- Active manifest definition parts now produce schema-validated canonical Definition packet envelopes, and those envelopes are grouped into one canonical Bundle.packet_set definition profile inventory for reseed readiness.
- The seeded definition profile audit compares packet material back to the core manifest and fails closed on missing parts, unexpected parts, stale profile metadata, duplicate bundle refs, or digest drift.
- Stored Definition and Bundle packet material is now reseed truth material, but execution remains trusted-local; imported definition packets may describe schemas, operations, policies, dependencies, planners, and builders, but they cannot introduce server code.
- Definition, Bundle, and Preference are now canonical packet types with body schemas, compatibility entries, builder support, pipeline inventory coverage, and packet-store validation like other active packet types.
- Claimed shell preference writes now run through the standard signed mutation prepare/finalize corridor as preference.element.set, preserving projected responses, actor proof/session checks, same-value no-op behavior, and legacy cache sync. /api/nexus/shell-preferences remains guest compatibility state only.
- The direct preference.element.interface.set runtime connector is retained as a definition/internal comparison bridge rather than the live claimed-write corridor.